
Desenvolvimento de Aplicações Web
Projeto Final - Jogo Web de Gestão de Recursos

Ano Letivo: 2025/2026

1 Enquadramento e Objetivo Geral

No contexto do Desenvolvimento de Aplicações Web, a capacidade de integrar frontend interativo com backend robusto em contexto de aplicação funcional e escalável constitui uma competência essencial para futuros profissionais de Tecnologias de Informação.

O presente projeto tem como objetivo consolidar os conhecimentos trabalhados nos laboratórios nos componentes de programação JavaScript (DOM, estruturas de dados, persistência) e programação servidor em Python/Flask (MVC, autenticação, persistência em base de dados, lógica de negócio).

O projeto consiste no desenvolvimento de um **jogo web de gestão de recursos**, onde o utilizador constrói e gere um reino/aldeia/quinta/loja (tema livre), gerindo construções, recursos e possibilidades de interação com outros jogadores.

O tema é livre, mas a mecânica e requisitos técnicos são fixos. O grupo deve:

- Implementar um backend em Flask com base de dados
- Criar um frontend responsivo com HTML/CSS/JavaScript
- Implementar um sistema de autenticação básico
- Implementar a mecânica core do jogo: construções, recursos, tempo
- Documentar e comunicar o projeto de forma clara

A diferença fundamental em relação aos laboratórios é que o estudante assume o papel de programador full-stack completo: design da arquitetura, implementação backend, desenvolvimento frontend, testes e documentação.

2 Objetivos Específicos

Ao concluir este projeto, o grupo deverá demonstrar que é capaz de:

- Arquitetar uma aplicação web seguindo o padrão MVC
- Implementar um sistema de autenticação básico com gestão de sessões
- Desenhar um esquema de base de dados adequado aos requisitos
- Criar um backend em Flask que implemente a lógica de negócio do jogo
- Desenvolver um frontend dinâmico que comunique com o backend via requisições HTTP
- Gerir estados da aplicação (inventários, construções, utilizadores)
- Documentar a arquitetura, API e instruções de execução
- Apresentar e defender o trabalho realizado

3 Dimensão dos Grupos

Os grupos deverão ter no máximo 3 elementos, por se considerar o número adequado para assegurar equilíbrio entre partilha de trabalho, diversidade de perspetivas e capacidade de coordenação no desenvolvimento.

Grupos de 2 elementos poderão ser autorizados, assumindo o grupo o risco da redução de diversidade. Trabalhos individuais serão aceites mas são fortemente desaconselhados, tanto pela carga de trabalho como pela limitação de aprendizagem colaborativa.

4 Requisitos de Trabalho e Entregáveis

O trabalho deverá incluir os componentes indicados na Tabela 1.

Tabela 1: *Resumo dos componentes entregáveis do trabalho*

Componente	Conteúdo exigido	Formato
Código Backend	Flask, rotas, lógica, base de dados	Python (.py)
Código Frontend	HTML, CSS, JavaScript	HTML, CSS, JS
Base de Dados	Esquema de dados implementado	Formato adequado ao projeto
Documentação Técnica	API, arquitetura, instruções	PDF ou Mark-down
Relatório	Contextualização, decisões, resultados	PDF
Código Fonte	Código organizado em pastas/ficheiros	Pasta comprimida (.zip)

5 Especificações Técnicas

5.1 Mecânica Core do Jogo

5.1.1 Tema Livre

O tema é livre. Exemplos válidos:

- **Gestão de Aldeia:** construções (cabana, moinho, quartel), população, defesa
- **Tycoon de Loja:** produtos, balcões, atendimento, lucro
- **Gestão de Quinta:** campos, colheitas, animais, produção
- **Construção de Imobiliário:** lotes, casas, lucro com rendas
- **Espaço Sideral:** estações, recursos, pesquisa
- Qualquer outro tema desde que respeite os requisitos mecânicos

5.1.2 Requisitos Mecânicos Obrigatórios

1. Sistema de Recursos

- Pelo menos 2 tipos de recursos primários (ex: ouro e madeira, ou energia e população, etc.)
- Recursos começam com quantidades iniciais ao registar novo utilizador
- Interface clara mostrando quantidade atual em tempo real

2. Construções/Estruturas

- Mínimo de 3 tipos de construções distintos
- Cada construção tem: custo (em recursos), tempo de construção, funcionalidade
- Número adequado de slots de construção por utilizador (posições/locais onde construir)
- Cada slot só pode ter uma construção activa por vez

3. Mecânica de Tempo

- Construções tem tempo de construção (1 minuto a 10 minutos recomendado)
- Após conclusão, utilizador recebe notificação
- Construções podem ter "tarefas" que levam tempo e retornam bónus de recursos

4. Sistema de Tarefas/Ordens

- Cada construção pode receber uma ordem (ex: "Minerar", "Comerciar", "Defender")
- Tarefa leva X tempo (5 minutos a 1 hora)
- Ao completar, retorna bónus em recursos (quantidade fixa ou percentual)
- Utilizador deve ir à construção (carregar em botão) para recolher o retorno após conclusão
- Interface indica claramente o estado: processando, concluída, recolhida

5. Interface de Utilizador

- Dashboard principal mostrando estado da aldeia/quinta/loja
- Visualização clara das construções e respetivos slots
- Indicadores de progresso para construções e tarefas
- Histórico de ações recentes

5.2 Autenticação e Sistema de Utilizadores

5.2.1 Requisitos Mínimos

- **Registo:** formulário com username, email e password
- **Login:** autenticação com username e password
- **Logout:** encerramento de sessão
- **Persistência de Sessão:** manter utilizador autenticado entre visitas (cookies/sessões)
- **Proteção Básica:** validação de entrada, proteção contra SQL injection (usar ORM correto)

5.3 Exemplos de Superação dos Requisitos Mínimos

Para além do que é pedido no enunciado principal, o grupo pode **superar os mínimos** através de escolhas adicionais de qualidade, profundidade técnica ou abrangência funcional. Os exemplos seguintes **não são obrigatórios**, mas ilustram formas de atingir um nível de execução correspondente a desempenho superior.

5.3.1 Autenticação e Segurança

- Registo avançado com avatar, verificação de email ou seleção de dificuldade inicial
- Recuperação de conta / reposição de password
- Perfil de utilizador com estatísticas, histórico de realizações ou personalização
- Medidas adicionais de segurança, como 2FA, rate limiting no login ou hashing reforçado de passwords

5.3.2 Interação entre Jogadores

Uma das formas de superar os requisitos mínimos consiste em acrescentar mecanismos inter-jogadores. Exemplos possíveis:

1. Ataque/Conflito

- Utilizador pode atacar outro jogador
- Roubo de recursos (parcial) ou dano a construções
- Sistema simples de defesa (construção defensiva reduz dano)

2. Comércio/Troca

- Utilizador pode enviar proposta de troca para outro
- Troca de recursos por acordo mútuo

- Sistema de aceitação/rejeição

3. Aliança/Cooperação

- Utilizadores podem formar alianças/clãs
- Bónus cooperativos (ex: 5% mais recursos se em aliança)
- Chat ou mensagens entre membros

4. Ranking/Leaderboard

- Leaderboard público ordenado por pontuação (total de recursos acumulados, construções, etc.)
- Atualização em tempo real
- Badges para top 3 jogadores

5. Eventos Globais

- Evento único que afeta todos os jogadores (mudança de taxa de recursos, bonus temporal)
- Criado manualmente pelo admin ou automaticamente
- Mecânicas interjogadores mais ricas ou melhor integradas no fluxo principal do jogo
- Sistemas de aliança, comércio, ranking ou eventos globais com maior profundidade funcional
- Melhor equilíbrio entre competição e cooperação, com feedback claro para o utilizador

5.3.3 Qualidade Global da Solução

- Interface mais polida, intuitiva e consistente
- Organização técnica especialmente clara e modular
- Melhor cobertura de testes, validação de erros e robustez geral
- Funcionalidades adicionais que façam sentido no contexto do jogo e estejam bem integradas

5.4 Stack Técnico Obrigatório

5.4.1 Backend

- **Framework:** Flask (Python)
- **Arquitetura:** Padrão MVC
- **Base de Dados:** solução de persistência adequada ao projeto (ex: SQLite, PostgreSQL ou outra)
- **Acesso a Dados:** SQLAlchemy ou abordagem equivalente, quando aplicável
- **Autenticação:** Flask-Login com sessões básicas
- **Validação:** Validação de entrada em rotas (tipos, ranges, etc.)

5.4.2 Frontend

- **HTML:** HTML5 semântico
- **CSS:** CSS3 com flexbox/grid para responsividade
- **JavaScript:** JavaScript (sem dependências pesadas de frontend em versão base)
- **Responsividade:** Funciona em desktop, tablet e mobile
- **Atualizações:** Fetch API para requisições ao backend

5.4.3 Desenvolvimento

- **Controlo de Versão:** Git + GitHub/GitLab (recomendado para trabalho em equipa)
- **Ambiente:** Virtual environment (venv) ou Docker

6 Metodologia

O trabalho deverá refletir as etapas do desenvolvimento full-stack web:

Planear → Design → Backend → Frontend → Integração → Testes → Deploy

O desenvolvimento será acompanhado em contexto de aula e parcialmente realizado em trabalho autónomo. Recomenda-se, fortemente, a divisão clara de tarefas e comunicação regular entre os membros do grupo.

6.1 Planeamento Sugerido

6.1.1 Fase 1: Planeamento e Design (Semana 1)

- Definir tema do jogo e mecânica detalhada
- Desenhar wireframes da interface
- Esquematizar base de dados
- Criar documento de design/vision

6.1.2 Fase 2: Backend (Semana 2-3)

- Implementar modelos de dados (Flask-SQLAlchemy)
- Criar rotas de autenticação (registo, login, logout)
- Implementar as rotas do backend do jogo (construções, recursos, tarefas)
- Implementar lógica de construções e recursos
- Implementar sistema de tempo (computar e atualizar)

6.1.3 Fase 3: Frontend (Semana 3-4)

- Criar layout responsivo (HTML/CSS)
- Implementar páginas de autenticação
- Criar dashboard principal do jogo
- Implementar componentes (construções, recursos, tarefas)
- Integração com requisições Fetch ao backend

6.1.4 Fase 4: Integração, Testes e Documentação (Semana 5)

- Testes de integração (frontend + backend)
- Correção de bugs e otimização
- Documentar as rotas do backend (formato e parâmetros esperados)
- Instruções de instalação e execução
- Preparar apresentação

7 Gestão e Organização

7.1 Prazo de Entrega

A entrega do trabalho final deve ser realizada na plataforma Moodle até ao dia **12 de junho de 2026, às 23:59**. É suficiente uma submissão por grupo.

7.2 Apresentação e Discussão Final

Na última semana de aulas, após a entrega, será realizada uma **apresentação e discussão** do projeto com todos os elementos do grupo.

Situação de Imprevisto

Caso ocorra algum imprevisto que impeça a entrega no prazo (problema de ligação à internet, problema técnico na plataforma Moodle, etc.), o grupo deve, **obrigatoriamente**, proceder à entrega física do projeto no **dia seguinte até às 12h00** na portaria da escola.

Instruções para entrega física:

1. Guardar todos os ficheiros (projeto, relatório, documentação) numa **pen USB** ou dispositivo de armazenamento externo equivalente
2. Entregar na **portaria da escola**
3. A pen deve estar claramente identificada com: **nome do grupo, disciplina (DAW), nome do projeto**
4. Indicar no envelope/rótulo: **"Endereçado ao responsável da disciplina"**

Importante: Esta via é apenas para situações de imprevisto genuíno. Atrasos sem justificação válida resultarão em penalização.

Conteúdo a entregar

Devem ser submetidos os seguintes ficheiros:

1. **projeto_nome_grupo.zip** – Código fonte completo (backend + frontend)
2. **relatorio_nome_grupo.pdf** – Relatório técnico (5–15 páginas)
3. **api_documentacao.md** – Documentação das rotas do backend
4. **README.md** – Instruções de instalação e execução
5. **base_dados.sql** – Esquema SQL da base de dados

Opcionalmente, pode ser incluído um **link para deploy** da aplicação funcional em servidor online (Heroku, PythonAnywhere, etc.).

7.3 Recomendações para Organização

Recomenda-se:

- Organizar o código em pastas/módulos claros
- Utilizar Git/GitHub (ou GitLab) para controlo de versão e partilha de ficheiros entre os membros do grupo
- Criar um `README.md` com instruções de instalação e execução
- Documentar as rotas do backend de forma clara
- Incluir um ficheiro com o esquema SQL ou instruções de criação da base de dados

Sugestões adicionais para melhorar a qualidade do trabalho serão fornecidas ao longo das semanas.

8 Critérios de Avaliação

O projeto será avaliado em cinco dimensões:

1. **Frontend** – Interface, usabilidade, organização visual, responsividade e integração com o backend.
2. **Backend** – Modelação de dados, implementação da lógica, rotas, autenticação e robustez do servidor.
3. **Extensão de Funcionalidades** – Alcance funcional do projeto, riqueza das mecânicas implementadas e integração das funcionalidades adicionais.
4. **Qualidade da Implementação** – Clareza do código, estrutura da solução, consistência técnica, validação e fiabilidade geral.
5. **Documentação** – Qualidade do README, instruções de execução, documentação técnica e clareza da comunicação do trabalho.

Cada uma destas dimensões será apreciada numa escala de **0 a 4 pontos**:

- **0** – Não cumpre o critério ou a evidência apresentada é insuficiente
- **1** – Cumprimento muito incompleto ou com falhas significativas
- **2** – Satisfaz os mínimos aceitáveis
- **3** – Cumpre com qualidade o que é pedido nesse ponto
- **4** – Supera claramente o que é pedido no enunciado nesse ponto

Os exemplos apresentados na secção de superação dos requisitos mínimos devem ser entendidos como **referências possíveis** para atingir um nível 4, e não como checklist obrigatória.

Nota: A classificação final resulta da apreciação global do trabalho entregue nestas dimensões e do desempenho individual de cada elemento na apresentação/discussão final. As classificações individuais podem ser diferentes dentro do mesmo grupo e, no limite, um estudante pode obter **0 valores** num projeto classificado com **20 valores** se não demonstrar conhecimento mínimo sobre o que foi implementado.

9 Checklist

Em seguida é apresentada uma checklist que pode auxiliar na verificação do progresso do trabalho.

- ☐ Tema do jogo definido e narrativa clara
- ☐ Documento de design criado (wireframes, mecânica)
- ☐ Esquema/modelo de dados desenhado
- ☐ Endpoints da API definidos
- ☐ Projeto Flask com estrutura MVC implementada
- ☐ Modelos de dados implementados (User, Building, Task, Resource, etc.)
- ☐ Autenticação (registo, login, logout) funcional
- ☐ Sistema de sessão implementado
- ☐ Lógica de construções funcionante (criar, destruir, listar)
- ☐ Sistema de recursos funcionante (ganhar, gastar, atualizar)
- ☐ Sistema de tempo implementado (tarefas completam após X tempo)
- ☐ Mecanismo de recolha manual de tarefas concluídas funcionante
- ☐ Rotas do backend funcionais e testadas
- ☐ Frontend HTML/CSS responsivo criado
- ☐ Dashboard principal funcional
- ☐ Integração Frontend/Backend com Fetch API
- ☐ (Opcional) Sistema interjogadores implementado

- ☐ README.md com instruções de instalação
- ☐ Rotas do backend documentadas
- ☐ Relatório técnico escrito (5–15 páginas)
- ☐ Apresentação preparada (slides ou demo)
- ☐ Aplicação testada em Firefox/Chrome/Safari
- ☐ Segurança: validação de entrada
- ☐ Performance: carregamento rápido, sem timeout
- ☐ Dados de exemplo criados (seed data)
- ☐ (Opcional) Código versionado em Git com histórico significativo

10 Resultado Esperado

O grupo deverá demonstrar uma aplicação web full-stack coerente, funcional e bem fundamentada, evidenciando domínio dos conceitos abordados na unidade curricular (autenticação, MVC, HTML/CSS/JavaScript) e capacidade de aplicar técnicas de engenharia de software (design, testes, documentação).

O trabalho deve refletir profissionalismo na abordagem ao problema, rigor técnico na implementação, segurança adequada e clareza na comunicação dos resultados.

O jogo deve ser **jogável, funcional e divertido**, mesmo que simples. Não é esperado um jogo de qualidade comercial, mas sim uma prova de conceito sólida.

Bom desenvolvimento!

11 Referências

- Flask Documentation. (2025). *Welcome to Flask*. <https://flask.palletsprojects.com/>
- <https://flask-sqlalchemy.palletsprojects.com/>
- MDN Web Docs. (2025). *Fetch API*. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- MDN Web Docs. (2025). *HTML*. <https://developer.mozilla.org/en-US/docs/Web/HTML>
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
- OWASP. (2025). *Top 10 Web Application Security Risks*. <https://owasp.org/www-project-top-ten/>